

Sistema de gestión de rutinas

Gonzalo Yuseff Niklitschek 21.546.049-5
Maximiliano Parra 21.076.202-7
Francisco Molinet 20.441.917-5
NRC: 13046
Vina del Mar 28-05-2026

1. Introducción

1.1 Motivación

En un centro de entrenamiento deportivo, los entrenadores necesitan una herramienta que les ayude a crear rutinas personalizadas para sus clientes de manera efectiva. La gestión manual de ejercicios, intensidades, y limitaciones de no repetición en semanas seguidas es susceptible a fallos y consume mucho tiempo.

La razón que impulsa este desarrollo proviene de la necesidad de tener un sistema organizado que haga más fácil la planificación del entrenamiento, eliminando la improvisación y garantizando una distribución equilibrada entre ejercicios de fuerza y cardiovasculares. Asimismo el proyecto tiene una finalidad educativa la cual es implementar principios de diseño de programación orientado a objetos, además de una arquitectura que distingue claramente la lógica de negocio de la interfaz de usuario.

1.2 Objetivos del trabajo

El objetivo principal del trabajo es ofrecer una herramienta que permita administrar un catálogo de ejercicios físicos y generar rutinas de entrenamiento personalizadas de forma automática, considerando criterios como el tipo de ejercicio, el nivel de intensidad y restricciones de uso recientes para evitar la repetición de actividades en semanas consecutivas.

Para cumplir con lo anterior se creara un sistema con interfaz gráfica de usuario en Java que pueda gestionar la aplicación de rutinas de ejercicio personalizadas, facilitando:

- Importar ejercicios desde un archivo CSV asegurando la validación del formato y la integridad de los datos.
- Crear rutinas teniendo en cuenta el tipo de ejercicio, la intensidad y la limitación de no repetir en semanas consecutivas.
- Examinar rutinas de ejercicio con una navegación total.
- Observar resúmenes estadísticos de las rutinas creadas.

El sistema debe llevar a cabo la separación entre backend y frontend, aplicar el patrón de suscripción-notificación y gestionar excepciones personalizadas.

1.3 Estructura del documento

Este informe se estructura en tres secciones fundamentales. La sección 2 detalla la solución técnica desarrollada, abarcando el modelo de clases, sus relaciones y las decisiones de diseño adaptadas. La sección 3 expone las conclusiones del estudio, analizando el logro de metas y los conocimientos adquiridos.

2. Descripción de la solución

2.1 Modelo propuesto

El sistema se organiza en dos paquetes esenciales que representan la división de tareas entre la lógica de negocios y la interfaz de usuario:

- **Paquete back:** Se encuentra la lógica de la aplicación, abarcando el modelo de datos, la persistencia, el procesamiento de rutinas y el sistema de eventos.
- **Paquete front:** Se encuentran las ventanas gráficas desarrolladas con Java Swing y el punto de entrada a la aplicación.

La comunicación entre ambos paquetes se realiza a través del patrón de diseño Observer (suscripción-notificación), el cual se implementa mediante las clases Event y Eventlistener. De esta forma, el backend puede decirle al frontend sobre cambios de estado sin generar acoplamiento directo entre las capas.

2.2 Diseño de clases

Paquete Backend

- **GestorAplicacion:** Orquestador central. Coordina carga de ejercicios, generación de rutinas, notificación de eventos y mantenimiento de estado.
- **Opciones:** Almacén de ejercicios en memoria mediante array estático de 100 elementos.
- **LectorCSV:** Clase utilitaria de lectura y validación de archivos CSV. Convierte líneas de texto en objetos Ejercicio.
- **Ejercicio:** Clase base del modelo. Representa un ejercicio genérico con atributos.
- **Cardiovascular:** Subclase de Ejercicio. Especialización para ejercicios de tipo cardiovascular.
- **Fuerza:** Subclase de Ejercicio. Especialización para ejercicios de tipo fuerza.
- **Rutina:** Contenedor de ejercicios seleccionados. Calcula tiempos totales y estadísticas por tipo e intensidad.
- **Event:** Objeto de transferencia para notificaciones. Encapsula tipo de evento y datos asociados.

- **EventListener:** Interfaz del patrón observer. Define el contrato para los suscriptores.

Manejo de excepciones

El Sistema necesita verificar muy bien los archivos CSV antes de aceptarlos. Para hacer esto, el sistema no permite la manipulación descontrolada de errores y utiliza excepciones dirigidas.

- **ArchivoinexistenteException:** El archivo CSV especificado no existe en la ruta indicada.
- **FormatoIncorrectoException:** Los datos no cumplen el formato esperado.
- **InformacionIncompletaException:** Faltan campos obligatorios o hay valores vacios en el CSV.

Paquete Frontend

- **Main:** Punto de entrada. Ventana principal que tiene menu de opciones y área de información. implementa EventListener.
- **VentanaCargarArchivo:** Dialogo modal para selección de archivo CSV. Muestra el progreso y el resultado de la carga.
- **VentanaGenerarRutina:** Formulario para configurar los parámetros de la rutina como cantidad por tipo y distribución por intensidad.
- **VentanaRevisionRutina:** Visualización ejercicio por ejercicio con navegación anterior/siguiente.
- **VentanaResumenRutina:** Panel de estadísticas agregadas como totales y por nivel de intensidad.

Relaciones entre clases

- **Herencia:** Cardiovascular y Fuerza extienden de Ejercicio. Reutiliza codigo y métodos base.
- **Composición:** Rutina contiene una colección de Ejercicio. GestorAplicacion contiene una instancia de Opciones y opcionalmente una Rutina.
- **Observación:** main, VentanaCargarArchivo y VentanaGenerarRutina implementan EventListener y se suscriben a GestorAplicacion.
- **Dependencia:** GestorAplicacion depende de LectorCSV para la carga inicial, pero no mantiene referencia permanente.

2.3 Justificación de decisiones

Las decisiones de diseño tomadas a lo largo del proyecto responden a principios fundamentales de la programación orientada a objetos, buscando mantener el código organizado, legible y fácil de mantener.

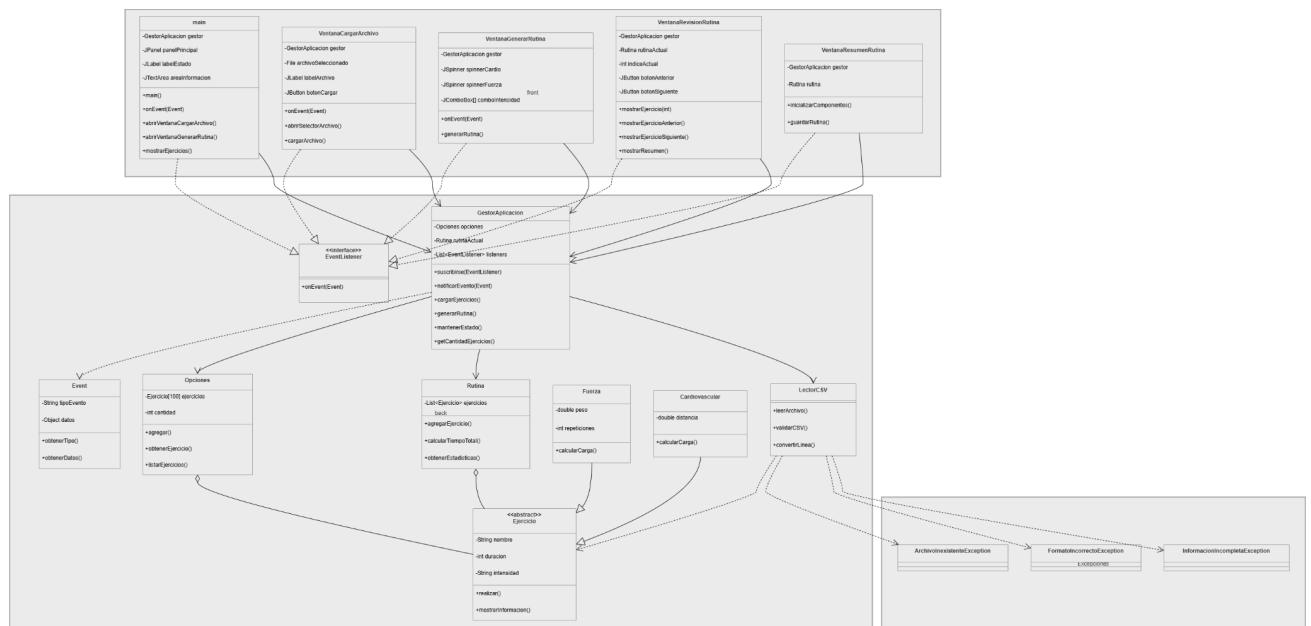
La separación del sistema en dos paquetes bien definidos permite que cada capa evolucione de forma independiente: los cambios en la interfaz gráfica no afectan la lógica de negocio y viceversa. En esa misma línea, aislar la lectura del CSV en `LectorCSV` responde al principio de responsabilidad única, permitiendo que `GestorAplicacion` se enfoque exclusivamente en orquestar el sistema sin preocuparse por los detalles del formato de archivo.

La jerarquía de herencia entre `Ejercicio`, `Cardiovascular` y `Fuerza` evita la duplicación de atributos y lógica común, y permite aprovechar el polimorfismo al filtrar ejercicios por tipo, eliminando la necesidad de condicionales basadas en texto a lo largo del código.

El uso de excepciones específicas para la validación del CSV permiten entregar mensajes de error precisos al usuario y facilita la identificación de problemas durante el desarrollo, al que con excepciones genéricas serían considerablemente más difíciles.

Por último, dividir el frontend en ventanas con responsabilidades acotadas reduce el riesgo de que modificaciones en una pantalla introduzcan errores en otra, haciendo el sistema más robusto frente a cambios futuros.

2.4 Diagrama de clases UML



3.1 Reflexión

Este proyecto dejó en evidencia que las decisiones de diseño tomadas en etapas tempranas tienen un impacto directo y duradero en la calidad del sistema. Elegir patrones adecuados, definir con claridad las responsabilidades de cada clase y anticipar los puntos de extensión son habilidades que este trabajo ayudó a desarrollar y que resultan fundamentales en cualquier proyecto de software de mediana o mayor escala.

A futuro, una mejora significativa sería incorporar un sistema de gestión de usuarios que permita a cada persona mantener un perfil propio con su historial de rutinas, nivel de condición física y progreso a lo largo del tiempo. Esto transformaría el sistema de una herramienta de uso puntual a una aplicación de seguimiento continuo, donde las rutinas generadas no sólo consideren el tipo de intensidad del ejercicio, sino también la evolución del usuario semana a semana, haciendo las recomendaciones progresivamente más personalizadas y efectivas.

3.2 Cumplimiento de objetivos

El desarrollo de este sistema de gestión de rutinas de entrenamiento permitió abordar de manera práctica los desafíos propios del diseño de programación orientada a objetos, integrando desde el modelado del dominio hasta la coordinación entre capas mediante patrones de diseño reconocidos. En términos generales, los objetivos planteados al inicio

del proyecto fueron cumplidos: el sistema es capaz de cargar un catálogo de ejercicios desde un archivo csv, validar su contenido, y generar rutinas personalizadas respetando las restricciones de tipo, intensidad y uso reciente.

3.3 Aprendizajes obtenidos

Uno de los aprendizajes más significativos del trabajo fue la aplicación del patrón Observer para desacoplar el backend del frontend. Esta decisión de diseño, materializada en la interfaz `EventListener` y la clase `Event`, demostró su valor al permitir que la lógica de negocios notifique cambios de estado sin depender de ninguna implementación concreta de la interfaz del usuario. De manera similar, la separación de responsabilidades entre clases como `LectorCSV`, `GestorAplicacion` y `GestorEjercicios` evidenció con una buena distribución de tareas facilita tanto la lectura del código como su mantenimiento futuro.

La jerarquía de herencia entre `Ejercicios`, `Cardiovascular` y `Fuerza` también representó un ejercicio valioso en el uso del polimorfismo, permitiendo que el sistema filtre y procese ejercicios según su tipo real sin necesidad de condiciones explícitas en cada punto del código. Por otro lado, la coexistencia de dos enfoques para el almacenamiento de ejercicios el array estático de `Opciones` y el `ArrayList` dinámico de `GestorEjercicios` permitió comparar en la práctica las ventajas de cada estrategia, siendo la segunda claramente más flexible y robusta para un sistema en crecimiento.